



DESIGN SYSTEM · V1.8

Octabase UI Style Guide

A consistent, modern Material Design 3 language for the Octabase project-management application — shared tokens, a unified icon system, components, and the rules for both surfaces: the full desktop app and the phone companion.

Foundation

Material Design 3

Typeface

IBM Plex Sans

Palette

Forest green · light · dark · octopus

Surfaces

Desktop app · Mobile companion

✓ Design Principles

1 **Consistent before clever.** One token, one component, one pattern per job. Two places that do the same thing look and behave identically.

2 **Icons carry recurring actions.** Close, edit, delete, add, refresh, filter, menu and friends render as icon buttons with an accessible label + tooltip — not text buttons. Text buttons are reserved for primary, page-level, or ambiguous actions.

3 **Modern & calm.** Generous whitespace on a 4-pt grid, soft elevation, fully rounded interactive surfaces, a restrained green palette.

4 **Two surfaces, one language.** The *desktop app* serves laptops, desktops and tablets (incl. iPad); *phones* are routed server-side to a purpose-built *mobile companion*. Both share the same tokens, palette, icon set and backend — they differ in layout, not in look. A *narrowed desktop window* still runs the full app with a collapsible drawer; only the board scrolls horizontally by design.

The one deliberate exception — wording, not meaning (resolved 2026-07-30 — this guide said the surfaces “differ in layout, not in look” and said nothing about text, while the mobile companion had already been shortening labels): the mobile companion *may* use a shorter label where the desktop wording does not fit a phone cell, and must do it through *its own key* in `octbase-mobile/locales` — never by editing a shared key. Shipped examples: the Vocabulary setting reads “Classic project management” on desktop and “Classic” on mobile; enabling MFA reads “Enable two-factor authentication” on desktop and “Enable 2FA” on mobile; the notifications tab is `nav.notifications` (German “Nachrichten”) while the view title stays “Benachrichtigungen”. The limit is strict: **the short form must mean the same thing** — abbreviate or drop a qualifier, never rename the concept, and never let the two surfaces use different vocabulary for the same object. Both locales move together, and the change gets a changelog entry like any other user-visible text.

5 **Accessible — WCAG 2.2 AA.** Every icon-only control carries `aria-label` + `title`; focus is always visible; hit targets ≥ 40 px for standalone and icon-only controls; color is never the only signal.

The one deliberate exception (resolved 2026-07-29 — this guide used to mandate ≥ 40 px while also defining a 32 px `sm` button, and said nothing about which won): the `sm` · 32 px button is permitted *inline in tables and dense rows*, where a 40 px control would break the row rhythm. It still clears WCAG 2.2 AA, whose *Target Size (Minimum)* threshold is 24×24 px. Everywhere else — toolbars, dialogs, anything icon-only — the 40 px floor applies, and `.icon-btn` carries it as an explicit `min-width` / `min-height` rather than leaving it to padding to add up. On phones the floor is higher still: `--touch` · 2.75 rem (≥ 44 px).

Color

A forest-green M3 palette. Use semantic tokens — never raw hex — in CSS rules and generated markup.

Key roles

Primary Primary #006C4F	Container Primary container #B0EDD0	Secondary Secondary container #CCE8D6	Tertiary Tertiary container #E3E9BE
Error Error #BA1A1A	Error c. Error container #FFDAD6	Background Background #F6FAF6	On-surface On-surface #191D19

Surface containers (white → green-tinted)

cl #FFFFFF	c-low #F0F5F0	c #EAF0EA	c-highest #DFE5DF
----------------------------------	-------------------------------------	---------------------------------	---

Dark theme

Both apps ship the matching M3 **dark palette** for the same green seed. It applies automatically from the OS (`@media (prefers-color-scheme:dark)`) and can be pinned with `:root[data-theme="dark"|"light"]` . The token *names* are identical — only their values flip — so semantic markup needs no theme branching.

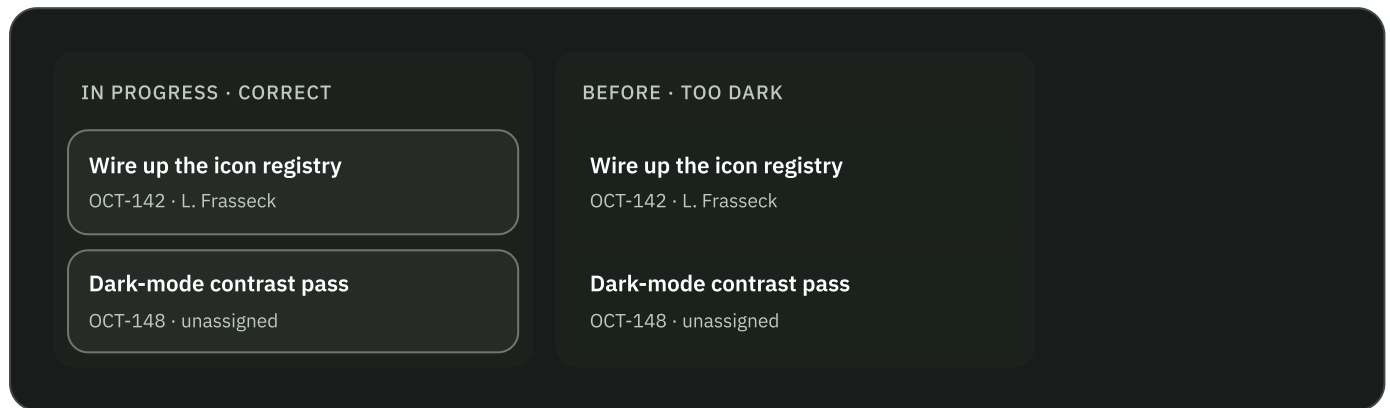
No flash of the wrong theme. A pinned theme is read from `localStorage` and stamped onto `<html>` by `js/theme-init.js` , which is loaded **synchronously in `<head>` , before the stylesheet** — so the first paint is already correct. Two constraints follow and both are load-bearing: it must stay a **classic, non- defer , non-module** script (module scripts are always deferred, which reintroduces the flash), and it must stay an **external file rather than inline**, so the Caddy CSP can remain `script-src 'self'` without `'unsafe-inline'` . Both SPAs share the `octbase-theme` key, since they are served from one origin (`/` and `/m/`).

Primary text (`--md-on-surface` / `--md-on-background`) is **pure white** `#FFFFFF` — deliberately brighter than M3's stock near-white, lifting body text from `~13:1` to `~17:1` on the `#191C1A` background for a crisper dark reading experience. Muted/secondary text stays `--md-on-surface-variant` `#BFC9C1` (`~9:1`), which preserves the primary/secondary hierarchy precisely because primary text moved further away from it.

Primary Primary #7ADCAD	Container Primary container #005138	Background Background #191C1A · text #FFFFFF	Surface cl Surface (cl) #1B211D · text #FFFFFF
--	--	---	---

Cards on a surface (dark)

Dark surface containers are clustered in luminance, so the light-mode pattern — a brighter fill plus a drop shadow — no longer separates a card from the lane it sits on (the shadow disappears on a dark background). Board/task cards in dark mode therefore use a **lighter container** (`--md-surface-c-high` `#272B27`) and a **1 px muted-outline border** (`#6D7870`). The fill difference alone can't reach 3:1 between adjacent dark surfaces, so the border carries it — `#6D7870` is **3.5:1** on the lane and **3.1:1** on the card fill, clearing **WCAG 2.1 SC 1.4.11** (Non-text Contrast, $\geq 3:1$) on both edges while staying intentionally subtle. The same border keeps light-mode external/read-only cards visible, since those share the lane fill.



Octopus theme

An optional third, light-style theme keyed to Octbase's octopus mascot — a warm coral/salmon palette sampled from the painted artwork that used to be the mascot (coral-red body, orange highlights, blush-pink suckers), kept as `img/octopus_small_original.png`. Unlike dark mode it **never engages from the OS preference**: it applies only when the user explicitly cycles to it (`:root[data-theme="octopus"]`). It overrides the same semantic token set as the dark block, so markup needs no theme branching.

Primary	Container	Background	Surface c
Primary #C03A2F	Primary container #FFDAD3	Background #FFF8F6	Surface (c) #FCEBE6

Octopus mascot

The decorative octopus in the bottom-right corner of the viewport is `img/octopus_small.png` — a stylized line-art octopus, black on transparency. It replaced the photo-realistic coral painting that used to hold that filename (kept as `img/octopus_small_original.png`, since the Octopus palette above was sampled from it), and it is drawn in **every** theme rather than only under `[data-theme="octopus"]`, which is what the full-colour painting had constrained it to.

Because the source is a **single colour**, one theme rule covers the whole problem: `filter: invert(1)` flips it to white on Dark, where black line work would otherwise sit invisibly on a near-black surface. That is exact for monochrome art and needs none of the `hue-rotate` chain the wordmark uses to reach a specific colour. It stays an `<img>` deliberately — a CSS `mask` would give per-theme tinting but silently renders nothing in the `file://` standalone bundle, because Chrome refuses CSS-referenced subresources across `file://` origins where an `<img>` is still allowed. Light and Octopus are both light-surfaced, so they take the artwork as it is.

It sits at `opacity:.85` , is `aria-hidden` with `pointer-events:none` , and is `position:fixed` below the interactive overlays, so it is invisible to assistive technology and can never intercept a click or cover a control.



Light
`--app-bg #f8f8f8`



Dark
`#12150F · invert(1)`



Octopus
`--app-bg #FBEDE9`

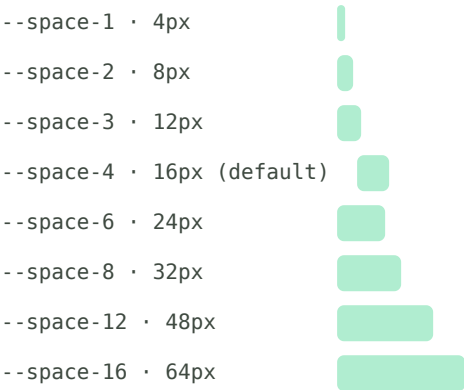
Typography

IBM Plex Sans, self-hosted (weights 100–700). Monospace stack for branch names, code and `kbd` .

1.5rem / 600	Page title — Sprint board
1rem / 600	Section title
0.875rem / 400	Body text — the default reading size for descriptions and content.
0.875rem / 500	Label / button
0.6875rem / 500	OVERLINE · CAPTION

Spacing & Shape

Spacing scale — 4-pt grid



Corner radius

Corrected 2026-07-29 against the shipped CSS. `lg` was previously labelled “dialogs” and `xl` “bottom sheet”; in practice `lg` is what the section panels use (release, sprint and project cards, dashboard and settings sections, the admin and audit lists), and **dialogs sit on `xl` alongside the bottom sheet** — which is also where Material 3 puts them. The task card is the `md` entry and was always conformant.

xs · 4 — type badges sm · 8 — inputs md · 12 — cards lg · 16 — section panels xl · 28 — dialogs, bottom sheet

full — buttons, chips, nav

Iconography

One coherent **outline icon set**, drawn from [Lucide](#) (ISC-licensed) — 24×24 viewBox, `stroke-width: 2`, round caps and joins, `currentColor`. It replaces the previous mix of emoji and Unicode glyphs. The official Lucide path data is vendored into a single **ICONS** registry (no runtime dependency, so the disk-served demo and offline builds keep working) and emitted in JS through one `icon(name, {size})` helper, so one glyph means one concept everywhere.

Three sizes, and only three: `sm` 16 px for glyphs sitting inline with text, `md` 20 px for controls, buttons and nav, `hero` 48 px for empty-state illustrations. Any other value — including a stray numeric px — is snapped to the nearest of these by `iconPx()`, so the set cannot drift back to a dozen ad-hoc sizes.

The grid below is **generated from the shipped ICONS registry** at page load, not transcribed from it: all 55 icons the app can render appear here, and a new one shows up the moment it is added to `js/icons.js`. Adding an icon to the app is therefore the whole job — this page cannot fall behind it.

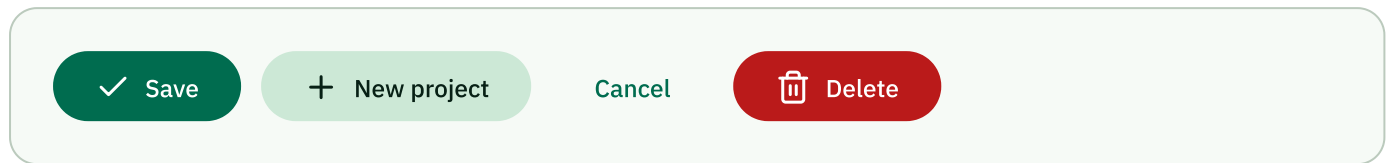
 add	 archive	 attach	 backlog
 bell	 board	 branch	 calendar
 check	 chevron-down	 chevron-left	 chevron-right
 chevron-up	 close	 collapse	 comment
 copy	 delete	 doc	 dot
 download	 drag	 edit	 expand
 external	 filter	 home	 image
 kebab	 link	 logout	 menu
 	 	 	

milestone	mindmap	more	page
 palette	 pin	 project	 refresh
 release	 save	 search	 settings
 sidebar	 sliders	 sort	 sprint
 stats	 theme	 time	 upload
 user	 view	 warning	

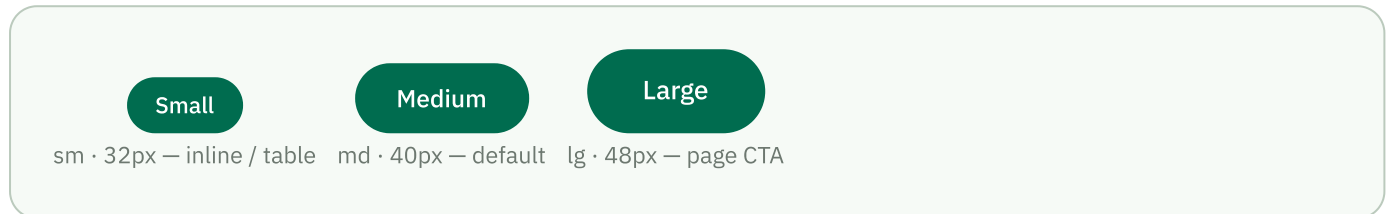
Each icon-only control pairs the glyph with an i18n-driven `aria-label` and `title` tooltip — never ship an icon without both.

Components

Buttons — for primary & ambiguous actions



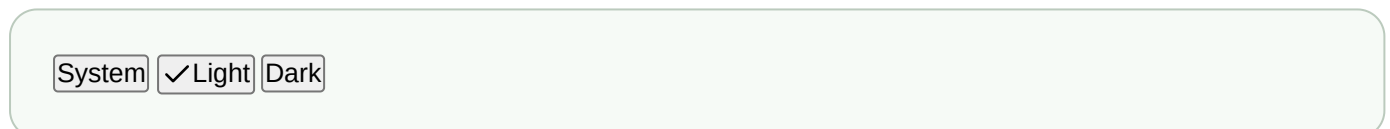
Button sizes — always content-sized, never full-width



Icon buttons — for recurring actions

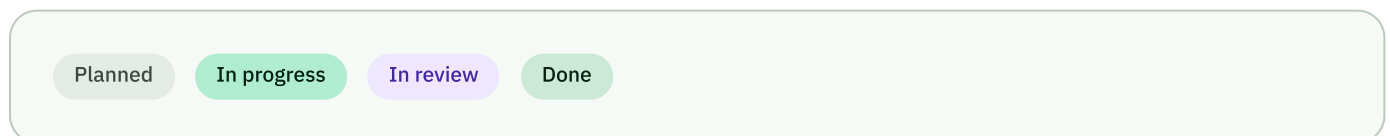


Segmented switch — for a small, mutually exclusive choice



`.seg-switch` — M3 segmented buttons: one `role="radiogroup"` of `role="radio"` buttons, 2–3 options. The selected segment gets `aria-checked="true"`, a check icon, and the secondary-container fill; unselected segments show only their label. Used for the Settings page's language, theme and vocabulary pickers — identically on desktop and mobile — and for the project's effort-estimation unit in **Task types & priorities**. This is the shared control for that kind of choice, not per-field dropdowns or bottom sheets; one implementation serves them all (`segSwitch()` in `js/framework.js`).

Status chips



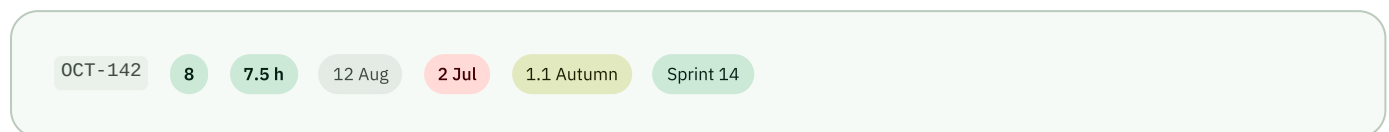
Priority badges — solid fill, never the only signal



The five built-in priorities are fixed; `prio-custom` (slate) renders any admin-defined project priority added under *Task types & priorities*. **BLOCKER** is more than a colour: a task cannot be moved to DONE while any of

its *open* child tasks still carries it (the API answers `422 TASK_HAS_BLOCKER`). A child that is already DONE or ARCHIVED does not count — a finished blocker blocks nothing, and counting it would strand the parent, since the guard would demand a priority change on a task whose priority it could no longer reach.

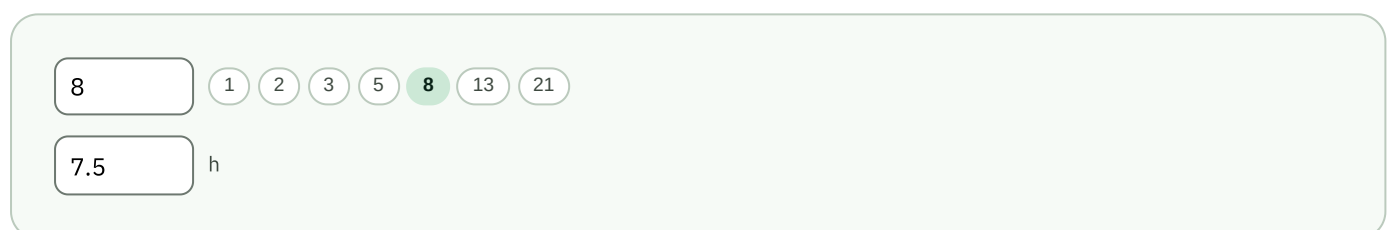
Card meta tags — pill-shaped, and each one optional



The meta row under a board card's title, reused verbatim on backlog and task-list rows. Every tag is **rendered only when it has a value** — none of them has an empty state, because an empty pill reads as a value of its own. That matters most for `estimate-tag`: a blank estimate would say *zero effort*, which is a different claim from *unestimated*, so an unestimated task simply has no tag. It is also absent entirely in a project that does not estimate, and on the container types (epic, initiative, theme) that cannot carry an estimate. Points render bare (`8`), hours carry their unit (`7.5 h`), both in tabular figures so a column of them aligns.

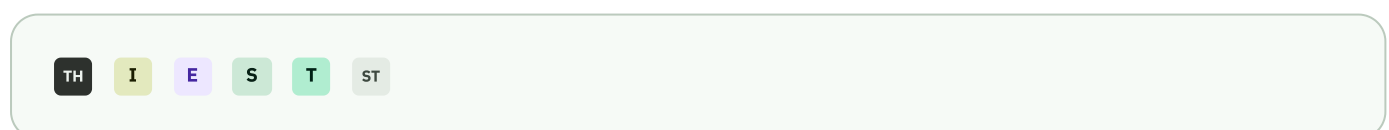
Colour is never the only signal here: `due-overdue` pairs the error container with a weight bump, and every tag carries a `title` naming what it is, since a bare number on a card is otherwise ambiguous.

Estimate editor — a free number field with shortcuts



The Fibonacci chips are **sugar over the input, not a scale**: the field accepts any whole number 0–100 (hours: 0–1000, two decimals), so a team on a different scale is never boxed in. Clicking the active chip clears the estimate, which is how the shortcut undoes itself. Hours get no chips — there is no conventional set — just the field and its unit suffix.

Work-item type chips



These six are the whole set. A work item is a theme, initiative, epic, story, task or subtask — there is no *bug* or *chore* type, and a defect is simply a task (usually a high-priority one). Earlier revisions of this guide defined `.type-bug` and `.type-chore` classes that the app has never shipped; they are gone, and the classes below are exactly what `app.css` carries.

A fixed 22 px square (`--md-xs` radius) carrying a bold one- or two-letter symbol, never the full word — the `title` tooltip (and an i18n-driven label elsewhere in the row) carries the full type name. The hierarchy runs theme → initiative → epic → story → task → subtask; *theme* and *initiative* are optional levels enabled per project. Inside the *Task types & priorities* settings modal the chip is enlarged to a 24 px square (`.ts-level` `.type-badge`) so the symbol stays legible alongside its toggle label.

Rich-text editor — one control, four places

The same editor renders the task description, the comment composer, the inline comment-edit box and the reply box. It is a `contenteditable` surface (`.rt-editor`) under a `.rt-toolbar` of `.rt-tool` icon buttons — the standard icon-button treatment, one glyph per command, `aria-label` and `title` on each.

Property	Value	Why
Height	<code>min-height: 7.2rem</code> , <code>max-height: 40rem</code> , <code>resize: vertical</code>	The drag handle lets a long comment be written at the height its writer wants rather than in a fixed well. The cap is raised, not removed, so pasted content still stops growing somewhere.
Focus	<code>outline: 2px solid --md-primary</code> , inset	Same focus ring as every other input; inset so it doesn't overlap the toolbar seam.
Drop target	<code>.rt-drop-active</code> — dashed primary outline + primary-container fill	Every editor accepts a dropped or pasted file, not just the description. The state must be visible before the drop, not after.
Empty state	<code>[data-placeholder]:empty::before</code>	<code>contenteditable</code> has no native placeholder.
Images	<code>max-width: 100%</code> , <code>--md-xs</code> radius	An inserted image renders as the picture while writing, identically to how it will render once posted.

The editor is available wherever text is authored, including on a **DONE or ARCHIVED** task's Comments tab: immutability freezes what a finished task *says* — title, description, type, dates, status — not the discussion about it.

Card · form field

In progress

⋮

Wire up the icon registry

OCT-142 · assigned to L. Frasseck

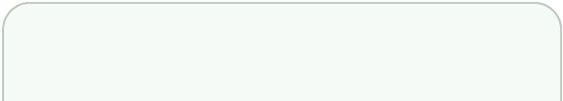
Project name

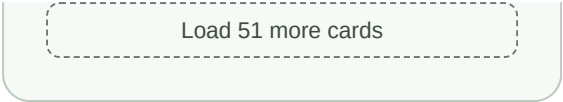
Octbase

Progressive disclosure — capping a long list

A list long enough to hurt is drawn a page at a time, and the rest arrives as the reader scrolls. Today that is the board lane (`.board-lane-more` , the project's `boardLaneLimit` , 20 by default); the pattern is the same wherever a container can grow without bound.

Two rules make the difference between paging and losing things. **The count never pages.** A lane's badge reports the whole lane, not the drawn part — a count that shrank with the render would say cards had been deleted. **The control is a button, not a bare sentinel.** Scroll position is how a pointer user reaches the rest, but it is not an affordance: a keyboard user needs something focusable and a screen-reader user needs it to say what is held back. So the sentinel *is* the button, and scrolling to it simply activates it.



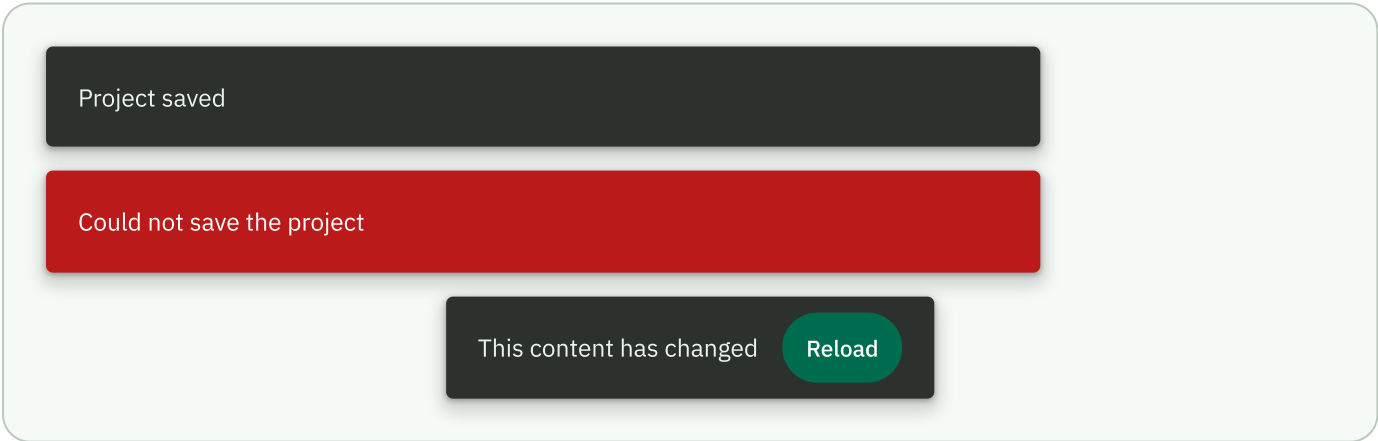


Rule	Applies
Dashed <code>1 px --md-outline</code> , centred, full lane width	Reads as a boundary in the list rather than as another item in it — it is not a card and must not look like one.
Names the remainder	“Load 51 more cards”, never a bare “Show more”: the number is the reader's only evidence that the count and the render disagree on purpose.
Disappears when exhausted	Its absence is the signal that the list is whole. It is never shown disabled.
Never hides a just-placed item	Something the reader created or dragged in is drawn wherever it lands, cap or no cap — otherwise the action appears to have done nothing.

Do Keep the container's count honest · say how many remain · let scrolling activate the same control a click would	Don't Page the count along with the render · use an invisible scroll sentinel with no focusable equivalent · cap a list so hard that a new item lands out of sight
--	--

Transient messages — snackbar and banner

Two shapes, and the choice between them is **who has to act**. A *snackbar* reports something that already happened and needs nobody; a *banner* reports a condition that persists until the reader decides. Neither is ever the only place an outcome appears — a failed save must also leave the form in its failed state.



Rule	Snackbar <code>.toast</code>	Banner <code>.content-stale-bar</code>
Purpose	Confirms an action that is already done, or reports that it failed.	A background condition the reader must resolve — today: someone else changed what is on screen.
Position	Bottom centre, stacked, <code>slideUp</code> 200 ms.	Below the top bar, centred, <code>z-index</code> under dialogs.

Dismissal	Automatic — 3.5 s. Never carries the only copy of anything.	Never automatic. It stays until its action is taken or the condition clears.
Actions	None. A snackbar the reader must click is a banner.	Exactly one, and it is the resolution — <i>Reload</i> .
Colour	Inverse surface; <code>--md-error</code> only for failures.	Inverse surface. Not error — stale is not broken.
Shape	<code>--md-xs</code> · 4 px and <code>--md-e3</code> . Deliberately the flattest radius in the system: these float over content and must not read as cards.	
Announcement	The container is a live region. Errors are <code>aria-live="assertive"</code> , everything else <code>polite</code> (WCAG 4.1.3 Status Messages) — set per message, not once.	

Do

Say what happened in the user's words · keep it to one line · let a failure also show up where it happened

Don't

Put an action in a snackbar that vanishes in 3.5 s · use error red for a stale-content banner · stack more than one banner · report progress with either — that is what a spinner and `aria-busy` are for

Layout

The **card/box views** — dashboard, projects, repositories, sprints, releases, settings — lay their cards out in one shared **two-column grid** (`.grid-2col`), so those pages read consistently. Three kinds of view are deliberately outside it: the **board** keeps its horizontal kanban layout; the **list/table views** (backlog, task list, pages, activity, archive, mindmap) are full-width because a table has nothing to pair with; and the **statistics page** uses the metrics layout below.

Repository connections

Boxes share the row as two equal columns.

Add repository

Display name

Columns are `repeat(2, minmax(0, 1fr))` above the `1024px` (`64rem`) Expanded breakpoint and a single `1fr` below it, with `gap: space-4` and `align-items: start`. `minmax(0,1fr)` stops wide children (long titles, progress bars) from forcing overflow; `align-items:start` lets each box keep its natural height rather than stretching to match a taller neighbour. Section labels that must break a row (e.g. the sprint *Active / Planned / Completed* headings) span both columns with `grid-column: 1 / -1`.

One layout primitive for all of these views: add `class="grid-2col"` to the content wrapper. Don't hand-roll per-view column counts or cap a view's width below the others.

Metrics layout — the one documented exception

A **metrics page** — today the project statistics page, and anything built like it — is the one card view that does *not* use `.grid-2col`, because its content is not a column of boxes to be read in order but a set of independently sized readings. It uses two `auto-fit` grids instead:

Utility	Track	Carries
<code>.stat-tiles</code>	<code>repeat(auto-fit, minmax(9.5rem, 1fr))</code>	The headline KPI tiles. Small, uniform, read at a glance — forcing two per row would waste most of the width and push the charts below the fold.
<code>.stats-grid</code>	<code>repeat(auto-fit, minmax(22rem, 1fr))</code>	The chart cards. 22 rem is the width below which an axis stops being legible, so the track floor — not a column count — decides how many fit. Two columns at 1024 px, three on a wide desktop.

A chart that needs more than one column takes `.stats-card.wide`, the same escape hatch `.grid-2col` gives its section labels — but it is capped at **two columns**, not the full row. It falls back to `grid-column: 1 / -1` and lifts to `span 2` under a `@container (min-width: 44.75rem)` query (two 22 rem tracks plus the gap: exactly when the second column exists). The query is on the grid rather than the viewport because the sidebar collapses, so container width decides the column count. Beyond two columns a headline chart stops gaining information and just stretches its plot area into empty space, and the sibling cards fill the rest of the row instead. Both grids keep `align-items: start` so a tall chart never stretches its neighbours.

This exception is **scoped to metrics pages**, and it is the reason the "no three-column grids" rule below says *card/box* views. It is not a licence to `auto-fit` anywhere a two-column grid feels tight — if a new view wants it, it belongs in this table first. The scope also covers the system's **one sub-1024 media query**: at `≤ 40rem` the distribution rows narrow their label

column and the page tightens its padding (`.dist-row` / `.stats-page`) — a content tweak owned by this layout, not a chrome breakpoint, and the only place one is permitted.

Rules

Do

Use `.grid-2col` for card/box content views · use `.stat-tiles` / `.stats-grid` for a metrics page · let the grid size the forms (a form sits in one half-width column) · keep boxes at natural height with `align-items:start` .

Don't

Full-width (100%) forms · three-or-more-column grids in a card/box view · a fresh hand-rolled `grid-template-columns` when one of the three utilities fits · ad-hoc `max-width` caps that make one view narrower than its siblings · a two-column grid on the board.

Charts & data display

Octbase draws the sprint burndown, the velocity trend and the statistics page. Charts are **hand-rolled inline SVG** — no chart library, no runtime dependency, consistent with the rest of the stack. They share one set of `.report-*` marks so the sprint report and the statistics page read as one system rather than two.

The colour rule — one hue per chart

This is the rule that constrains everything else, and it is not a preference. The app ships **four themes** (light, dark, octopus, and the system-followed pair), and each one **redefines** `--md-primary`. A hard-coded categorical palette — one colour per status, one per type — would be tuned against exactly one of them and would clash or fail contrast in the other three.

So: **every chart is a single series in a single hue** (`--md-primary`), and categories are told apart by their **row or axis label**, never by colour. The only charts that carry two series are the burndown (actual vs. ideal) and the velocity (committed vs. completed) — and both separate the second series by **texture and weight** rather than hue: the ideal line is a dashed `--md-outline`, the committed bar is the muted `--md-outline-variant` behind the solid primary. Those two, and only those two, get a `.report-legend`.

Do
One hue per chart, from a theme token · distinguish series by dash, weight or fill density · label every category in text · let a legend appear only where a second series exists.

Don't
A per-status or per-type colour scale · raw hex in a mark · colour as the only difference between two things · a legend on a single-series chart (the labels already say it).

Marks

Class	Role
<code>.report-grid</code>	Horizontal gridlines — <code>--md-outline-variant</code> at 50 % opacity. Behind the data, never competing with it.
<code>.report-axis-label</code>	10 px <code>--md-on-surface-variant</code> . The smallest type in the system; nothing else may go below it.
<code>.report-line-actual</code> · <code>-ideal</code>	Solid primary vs. dashed outline. The dash <i>is</i> the distinction.
<code>.report-bar-completed</code> · <code>-committed</code>	Solid primary vs. muted outline-variant, <code>rx: 2</code> .
<code>.report-legend</code> · <code>.report-swatch</code>	Two-series charts only.
<code>.report-note</code>	The caption under a chart: sample size, an unestimated-task count, an overdue warning. Where a chart's honesty caveats live.
<code>.dist-track</code>	The horizontal magnitude bar used for distributions (status, type, priority, per-person load) — a labelled row, not a pie.

Accessibility & honesty

1 **Every chart is** `role="img"` **with an** `aria-label` that states what it plots. A screen reader gets the sentence, not the geometry.

2 **Every mark carries a** `<title>` so hovering a bar or point reads out its value. A chart you can only estimate by eye is a decoration.

3 **A zero is a zero, not a gap.** A quiet week is a zero-height bar that still occupies its slot — dropping it silently rewrites the trend.

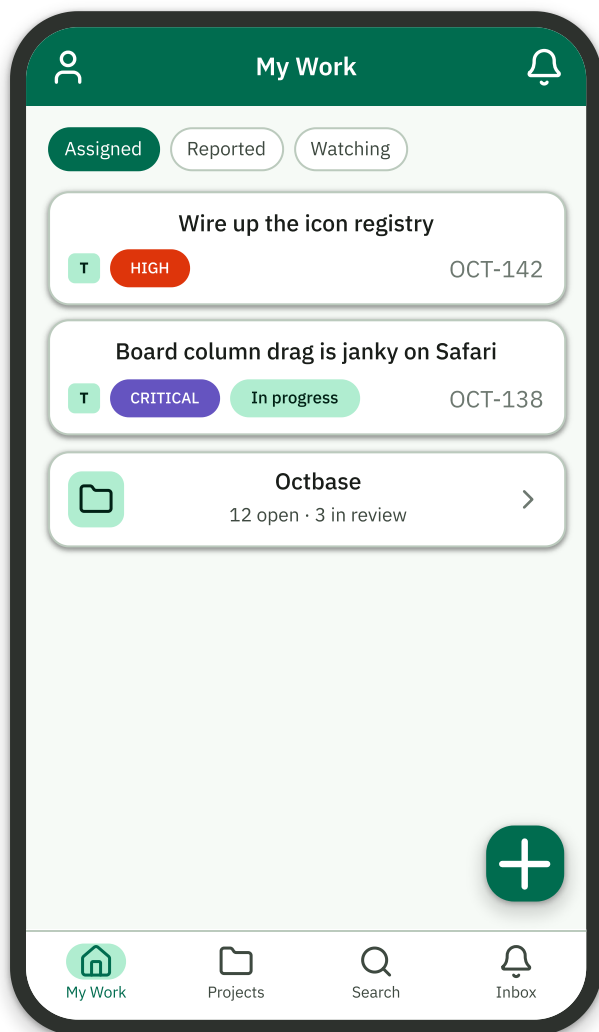
4 **Say what is missing.** Where a projection is incomplete — committed tasks with no estimate, a 90-day cycle-time sample — the count goes in the `.report-note` rather than being quietly folded in as zero.

5 **Median beside average** wherever one straggler can drag a mean somewhere no real item lives (cycle time). Never the average alone.

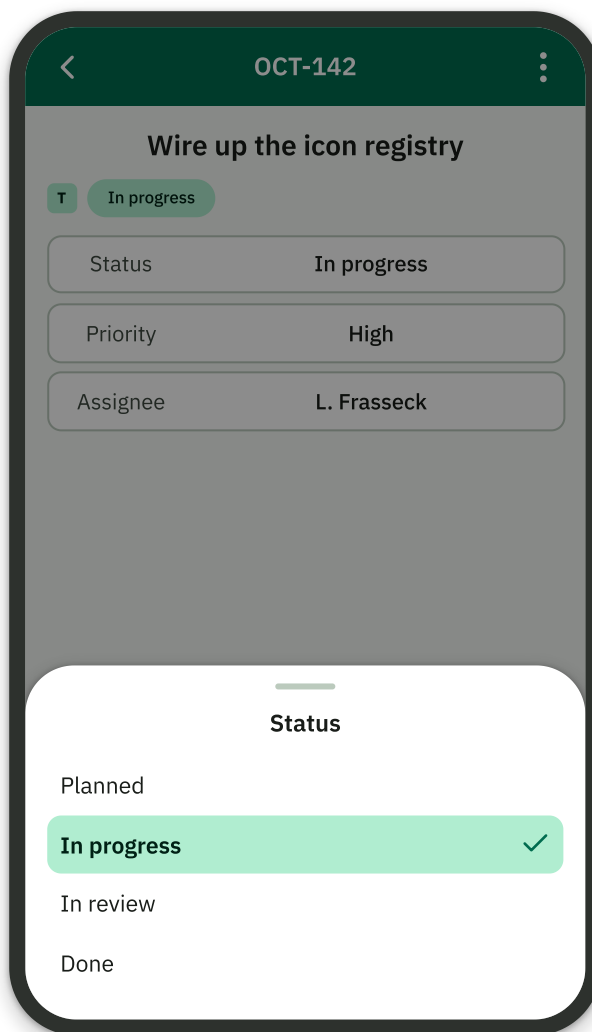
A new chart inherits all of the above by using the `.report-*` marks. If it seems to need a second hue, it almost certainly needs a second chart, or a labelled row per category.

Mobile companion

Phones get a **purpose-built companion app** (served at `/m/`) — not a squeezed desktop. It reuses every shared token (color, type, spacing, radius, elevation) and the same icon set, but the layout is authored mobile-first for one-handed use on a 360–430 px phone. It is an **enhancement, not a replacement**: it covers the day-to-day flows (My Work, projects, board, backlog, task detail, comments, create, search, notifications) and hands the rest off to desktop rather than cramming it in.



My Work · top app bar, FAB, bottom nav



Task detail · property rows + bottom sheet

Mobile patterns

Pattern	Token / rule	Notes
Top app bar	<code>--app-bar-h: 3.5rem + --safe-top</code>	Sticky, primary-colored; lead = back or profile, trailing = one context action.
Bottom nav	<code>--bottom-nav-h: 3.75rem + --safe-bottom</code>	Four destinations — My Work · Projects · Search · Inbox; active tab gets a primary-container pill behind the icon.
FAB	<code>3.5rem</code> , radius <code>--md-lg</code>	Single primary create action, floats clear of the bottom nav.

Bottom sheet	<code>max-height: 85dvh</code> , scrim <code>.45</code>	Replaces desktop dropdowns/dialogs for pick-one choices (status, priority, assignee).
Filter chips	<code>.seg-scroll</code> (sticky, h-scroll)	Board column switcher & list filters; active chip is solid primary.
Touch targets	<code>--touch: 2.75rem</code> ($\geq 44\text{px}$)	Every interactive row, chip and icon button clears the minimum.
Safe areas	<code>env(safe-area-inset-*)</code>	App bar and bottom nav reserve the notch / home-indicator insets.
iOS zoom guard	inputs at <code>font-size: 1rem</code>	$\geq 16\text{px}$ text stops Mobile Safari auto-zooming on focus.

On tablet/desktop widths the mobile app doesn't stretch edge-to-edge — it presents as a **centered ~430px phone frame** (`@media (min-width: 768px)`), so it stays usable when previewed or opened directly.

Rules

Do

Reuse the shared tokens & icon set · keep targets $\geq$ `--touch` · use bottom sheets for pick-one · offer “*Open on desktop*” for anything the phone omits — never a dead end.

Don't

Port desktop dialogs/drawers verbatim · add a 5th+ bottom-nav tab · build phone-only colors or type · hide an omitted feature with no hand-off link.

Responsive

Routing is **device-based, decided server-side** at the desktop app's Caddy front door — not by a CSS breakpoint. A **phone** User-Agent hitting `/` gets a `302` to the mobile companion under `/m/`; everything else — desktops, laptops and **tablets incl. iPad** (their UA omits the `Mobile` token) — stays on the full desktop app. `/api/*` is never redirected, so both surfaces share the same origin + JWT cookies. Within the desktop app a single `1024px` (`64rem`) breakpoint is all the chrome needs; a narrowed window keeps the full app and never collapses into a phone layout. (The metrics layout's `≤ 40rem` content tweak is the one documented exception — see *Metrics layout*.)

Context	Surface	Layout
Phone UA matches iPhone / Android-mobile / ...	Mobile companion (<code>/m/</code>)	Purpose-built one-handed app — bottom nav, FAB, bottom sheets, full-screen task detail (see <i>Mobile companion</i>).
Narrow desktop < 1024px (laptop / tablet)	Desktop app (<code>/</code>)	Off-canvas drawer + scrim, hamburger toggle in the app bar; card/box views collapse to one column; the board scrolls horizontally.
Desktop ≥ 1024px	Desktop app (<code>/</code>)	Permanent drawer; full multi-column layout; card/box views use the two-column grid.

Two-column content grid

Every card/box view except the board — dashboard, projects, repositories, sprints, releases, settings — lays its sections out with the shared `.grid-2col` utility: two equal `minmax(0,1fr)` columns at `≥ 1024px`, collapsing to one column below it. `minmax(0,...)` stops long titles and progress bars from forcing overflow; `align-items:start` keeps each box at its natural height.

Dialogs & overlays

Every dialog (`.modal` , incl. confirm/delete) is sized `width:min(30rem, calc(100vw - 2·space-4))` (wide variant `38rem`) so it never exceeds the viewport nor sits edge-to-edge; the backdrop keeps `space-4` padding and is pinned to `100dvh` . Height is capped at `min(90vh, 100dvh - 2·space-4)` and only the body scrolls, so the `.modal-footer` actions stay pinned and reachable at any height. The command palette, shortcuts, notifications and login box scale the same way.

Task panel

Task details open in a fixed-width side sheet (`#task-panel` , sized to `--panel-w`) that slides in from the right over the content. Its tab strip (`.panel-tabs` — Details · Comments · Relations · Branches ...) keeps one consistent desktop look across tabs.

Rules

Do

Use `min()/clamp()/%` widths · `minmax(0,1fr)` grid tracks · `min-height:0` on scrolling flex children · the

Don't

Fixed pixel widths wider than the viewport · phone-specific layouts (mobile is redirected, not restyled) ·

single 1024 breakpoint · the shared `.grid-2col` for card/box views.

horizontal scroll outside the board · icon-only buttons without labels · ad-hoc breakpoints (600/767/800/900/1160).